



Tag 4: Inventar-App – Datenbereitstellung & Auswertung

Ziel der Übung

Erweitere deine **Inventar-App** aus Tag 3 (Domäne: **Person**, **Device**, **DeviceType**, **Location**, **Assignment**) so, dass

- die **Ausleihen (Assignment)** in einer gut auswertbaren Form bereitgestellt werden,
- Fachbereiche die Daten bequem in **Excel** über **CSV/XLSX + Power Query** nutzen können,
- das Datenmodell um einen **Schaden-Vermerk** bei der Rückgabe erweitert wird (Migration mit Alembic),
- ihr am Ende ein paar sinnvolle **Pivot-Auswertungen** zur Inventarnutzung zeigen könnt.

Abgabe (mündliche Demo)

Ihr zeigt in der Gruppe:

- einen Branch mit der erweiterten Inventar-App (z. B. **feat/tag4-inventar-export**),
- funktionierende Endpunkte für **GET /assignment.csv** und **GET /assignment.xlsx**,
- mindestens **zwei Pivot-Auswertungen** in Excel auf Basis eurer Daten,
- kurz, wie ihr eine Schemaänderung (z. B. Schaden-Vermerk) fachlich motiviert und technisch umsetzt.

A) Schemaänderung mit Alembic – Schaden-Vermerk ergänzen

Zeitbudget: ca. 30 Min.

Fachlicher Fokus: **Wie entwickle ich ein Schema sauber weiter?**

1. Fachliche Idee formulieren

Überlegt euch, **welche Information** ihr bei der Rückgabe festhalten wollt, z. B. „Gab es einen **Schaden** am Gerät?“ Wie kann eine kurze Beschreibung erfasst werden?

2. Attribut entwerfen

- Tabelle: `assignment`
- mögliches neues Attribut
- überlegt, ob das Feld **optional** sein soll (NULL erlaubt) oder Pflicht.

3. Migration anlegen

- orientiert euch am **Alembic-Beispiel im Vorlesungsskript**,
- erstellt eine neue Migration, die in `upgrade()` eine Spalte an der Tabelle `assignment` ergänzt und in `downgrade()` wieder entfernt.

4. Migration ausführen & prüfen

- Migration ausführen (`alembic upgrade head` o. ä.),

z. B. mit `psql` oder eurem DB-Tool prüfen, ob die Spalte existiert.

5. Kurze Notiz machen

Haltet stichpunktartig fest:

- warum ihr die neue Spalte braucht,
- welche Überlegungen ihr zu Name, Typ und Optionalität hattet.

B) Assignment-Exports per HTTP bereitstellen

Zeitbudget: ca. 60 Min.

Ziel: eine **Sicht auf die Ausleihen**, die alle relevanten Infos für Reporting enthält.

1. Spalten der Assignment-Sicht definieren

Überlegt in der Gruppe, welche Spalten ihr in einer Auswertung sinnvoll findet, z. B.:

- `assignment_id`
- Geräteinfos: `inventory_number`, `serial_number`, `device_type`
- Standort: `location_name`
- Person: `person_name`, ggf. `person_email`
- Zeit: `date_from`, `date_to`
- Schaden-Feld (`damage_notes`)

Notiert euch die Liste der Spalten in einer kleinen Skizze (z. B. auf Papier oder in `docs/domain.md`).

2. SQL-Join für Assignment-Sicht entwerfen

Leitet aus eurem ERD und der Spaltenliste eine passende SQL-Abfrage ab, die

- von `assignment` aus startet,
 - die Tabellen `device`, `devicetype`, `location` und `person` verbindet,
 - genau die Spalten liefert, die ihr unter 1. definiert habt.
 - **Endpoint `GET /assignments.csv` implementieren**
 - Legt einen neuen Endpoint an (z. B. in `main.py` oder einem passenden Modul),
 - verwendet eure Assignment-Abfrage,
 - gibt das Ergebnis als **CSV** zurück (Trennzeichen `;`, Header-Zeile, sinnvolle Reihenfolge).
- Tipp: Orientiert euch am `/grades.csv`-Beispiel in der Vorlesung, passt aber
- Routenpfad,
 - Spaltennamen,
 - und Query an eure Inventar-Sicht an.

3. Endpoint `GET /assignments.xlsx` implementieren

- baut zusätzlich einen Endpunkt, der dieselbe Assignment-Sicht als **Excel-Datei (XLSX)** bereitstellt,
- nutzt Pandas + `ExcelWriter`, wie im Vorlesungsskript bei `grades.xlsx` gezeigt,
- prüft in Excel, ob Datentypen und Spalten passen.

4. Kurz testen & dokumentieren

- Ruft beide Endpunkte im Browser oder per Tool auf (`/assignments.csv`, `/assignments.xlsx`),
- speichert die Dateien,
- notiert euch für die Abgabe kurz: „Was ist der Unterschied in der Nutzung von CSV vs. XLSX für unsere App?“

C) Excel-Pipeline mit Power Query aufbauen

Zeitbudget: ca. 45 Min.

Ziel: stabile **Datenpipeline** von der App nach Excel.

1. Variante 1 – Power Query mit CSV-URL

- legt eine neue Arbeitsmappe an (z. B. `inventar_assignment_url.xlsx`),
- Datenquelle: **Aus dem Web** → eure URL von `/assignments.csv`,
- stellt sicher, dass
 - * das Trennzeichen korrekt erkannt wird (;),
 - * die Datentypen sinnvoll gesetzt sind (Text/Datum etc.),
- speichert die Abfrage unter einem sprechenden Namen (z. B. `AssignmentFromApi`).

2. Variante 2 – Power Query mit XLSX-Datei

- legt eine zweite Arbeitsmappe an (z. B. `inventar_assignment_xlsx.xlsx`),

- Datenquelle: **Aus Arbeitsmappe** → die von eurer App generierte `assignments.xlsx`,
- importiert das passende Blatt,
- prüft und korrigiert Datentypen.

3. Vergleich der Varianten

Diskussion in der Gruppe:

- Wann ist die URL-Variante sinnvoller (z. B. „immer aktuelle Daten“)?
- Wann kann eine XLSX-Datei sinnvoll sein (z. B. monatliche Snapshots)?
- Welche Variante würdet ihr einem Fachbereich empfehlen – und warum?

Haltet eure Erkenntnisse in Stichpunkten fest (für die kurze Präsentation am Ende).

D) Pivot-Auswertungen zur Inventarnutzung

Zeitbudget: ca. 30 Min.

Jetzt werden aus den Daten **Fragestellungen** – und ihr beantwortet sie mit Pivot Tables.

1. Aktive Ausleihen pro Standort

- definiert, was für euch „aktiv“ bedeutet (z. B. `date_to` leer / in der Zukunft),
- erstellt eine Pivot, die zeigt: „Wie viele aktive Assignments pro `location_name`?“

2. Nutzung pro DeviceType

- Pivot mit `device_type` als Zeilen,
- als Wert z. B. Anzahl `assignment_id`,
- optional Filter auf Zeitraum (z. B. nur Assignments der letzten 12 Monate).

3. Geräte pro Person

- Pivot mit `person_name` als Zeilen,
- Wert: Anzahl aktiver Assignments,

- beantwortet die Frage: „Wer hat aktuell wie viele Geräte im Einsatz?“

4. **Eigene Fragestellung (optional)**

Überlegt euch eine weitere Auswertung, die in einem realen Unternehmen hilfreich wäre, z. B.:

- „Welche Locations haben die meisten Schäden gemeldet?“ (falls ihr das Schaden-Feld nutzt),
- „Welche DeviceTypes sind im Schnitt am längsten ausgeliehen?“

Setzt diese Auswertung mit Pivot um und notiert kurz, welche Geschäftsfrage sie beantwortet.

E) Vorbereitung auf die Präsentation

Bereitet zum Abschluss eine kurze mündliche Demo (5–10 Minuten) vor:

1. **API-Exports**

- zeigt kurz, wie ihr [/assignments.csv](#) und [/assignments.xlsx](#) aufruft,
- erklärt in 1–2 Sätzen, wie die dahinterliegende Assignment-Sicht aufgebaut ist.

2. **Excel-Auswertungen**

- zeigt 2–3 Pivot-Auswertungen,
- erklärt jeweils, welche praktische Frage sie beantworten.

3. **Reflexion (Stichpunkte)**

- was lief gut,
- wo hattet ihr Stolpersteine,
- welche Variante (CSV-URL vs. XLSX-Datei) würdet ihr in einem echten Projekt bevorzugen – und warum?